Unified Tenant Isolation and QoS Detection in Kubernetes

1st H S Jayanth
231IT024
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Suvarna G
231IT077
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Multi-tenancy meaning multiple customers sharing a single infrastructure pool is central to cloud computing, yet Kubernetes runtime detection tools address only half the problem. A shared cluster can fail tenants in two ways: an isolation violation (one tenant improperly accessing another's resources) and a fairness/QoS violation (one tenant's workload starving a co-located tenant's performance, the "noisy neighbor" problem). Existing tools handle only security misconfiguration and miss fairness entirely.**

This project builds a unified runtime detection pipeline covering both failure modes: a control-plane monitor (Kubernetes audit logs), a network-flow monitor (eBPF via Cilium/Hubble), and a resource monitor (kubelet/cAdvisor), fused by a deterministic classification layer. Resource pressure alone is insufficient to declare a fairness violation, victim-side QoS degradation (rolling p95 latency) must be independently confirmed, eliminating false positives from legitimate high-CPU workloads. Eight repeatable injection scenarios and a quantitative evaluation harness are implemented and smoke-tested against a static baseline (kube-bench).

Keywords: Kubernetes, multi-tenancy, tenant isolation, QoS fairness, noisy neighbor, eBPF, Cilium, runtime detection, network policy, resource monitoring

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Cloud computing, at its core, is about sharing. When you rent compute from AWS, GCP, or any cloud provider, you are not getting a dedicated machine. You are getting a slice of a much larger pool of infrastructure that dozens or hundreds of other customers are using at the same time. Kubernetes is the technology that makes this sharing work at scale, grouping workloads into namespaces and enforcing boundaries using RBAC policies, network rules, and resource quotas.

The problem is that these boundaries are only as good as the configuration behind them. And when they fail, they fail in two very different ways.

The first is a **security failure**, one tenant accidentally or maliciously accesses another tenant's data or resources. Maybe a misconfigured role binding gives a service account permissions it shouldn't have, or a network policy is missing and traffic flows where it shouldn't.

The second is a **performance failure**. One tenant's workload consumes far more CPU or memory than it's supposed to, and the tenant running next to it starts seeing slow response times and degraded service. No unauthorized access happened, no security rule was broken, but the affected tenant is still being harmed. This is the classic "noisy neighbor" problem, and most security tools completely ignore it.

Identify applicable funding agency here. If none, delete this.

II. Literature Survey

A. Base Paper: DAST for Kubernetes (FSE '25)

Shamim, Hu, and Rahman's industry experience report evaluates three Dynamic Application Security Testing (DAST) tools against a live Kubernetes deployment and finds thousands of CIS-recommended configuration violations that static analysis alone missed. Its central argument, that static checks cannot catch violations that only manifest once a cluster is running under real workloads, is correct and directly motivates this project. However, its scope is limited in several ways:

- **No implemented artifact:** the paper documents findings and recommendations but does not design, build, or release a detection tool.
- **Single-dimension notion of violation:** only security misconfiguration is considered; resource-fairness and noisy-neighbor failures are entirely out of scope.
- **No quantitative evaluation:** no precision, recall, false-positive rate, or detection latency is reported.
- **Single signal source:** dynamic detection relies on control-plane signals alone, missing violations visible only at the data or resource plane.

B. eBPF-Based Kubernetes Runtime Security

Tools such as Falco and Cilium/Tetragon provide general-purpose eBPF-based runtime anomaly detection for Kubernetes. Cilium, via its Hubble observability layer, is used directly in this project as the network-layer eBPF data source. While these tools are mature and effective, they treat runtime security generically. None frame detection around cross-tenant boundary violations, and none pair security detection with a resource-fairness signal.

C. Cloud Resource Fairness and the Noisy-Neighbor Problem

A separate body of work addresses cloud resource fairness and noisy-neighbor mitigation. This work is generally concerned with scheduling and enforcement (e.g., cgroup-based

throttling, Kubernetes QoS classes) rather than correlating fairness breaches against an independent victim-side performance signal, and it does not connect to tenant-isolation security detection in any way.

D. The Gap

No identified IEEE/ACM publication unifies tenant-isolation security detection and tenant fairness/QoS detection under a single fused pipeline, evaluated quantitatively across both failure modes. This is the gap this project fills.

III. Motivation

Three observations motivate this project directly.

Multi-tenancy is not a peripheral concern for cloud computing, it is the concern. A cloud provider's central operational question is not "is this cluster misconfigured?" but rather "are my tenants safely *and* fairly sharing this pool of infrastructure?" Existing tooling answers only the security half of that question.

Isolation and fairness violations share a root cause but are invisible to each other's detectors. A control-plane monitor sees who asked to access what, but is blind to a CPU-hogging batch job that never makes an unauthorized API call. An eBPF network monitor sees cross-namespace traffic, but not cgroup CPU throttling, as that data lives in an entirely different subsystem (kubelet/cAdvisor).

No one has quantitatively evaluated a unified pipeline for both. Without precision, recall, false-positive rate, and detection latency, measured against a structured ground-truth scenario library and a static baseline, claims about detection effectiveness remain anecdotal.

IV. Objectives

The project is organized around six objectives, built and validated in dependency order:

1) **Construct a multi-tenant Kubernetes testbed and establish a static configuration baseline.** Stand up a local kind cluster with Cilium/Hubble and Kubernetes audit logging; deploy a demo multi-tenant application across three tenant namespaces, each with its own RBAC, default-deny NetworkPolicy, and ResourceQuota/LimitRange; and run kube-bench as the static-only comparison point.
2) **Build a control-plane monitor for isolation violations.** Tail Kubernetes audit logs in near real time and flag any principal from one tenant's namespace accessing a resource scoped to a different tenant.
3) **Build data-plane and resource-plane monitors for isolation and fairness signals.** Wrap `hubble observe` to flag network flows crossing tenant namespace boundaries, and poll kubelet/cAdvisor for per-pod CPU throttling and memory pressure.
4) **Build a fusion and classification layer that unifies both tracks.** Correlate events from all three monitors by namespace pair and time window, classifying each anomaly as an isolation violation, a fairness violation, or both. Independent victim-side QoS degradation is required as evidence for any fairness classification.
5) **Add a CI/CD security gate.** Scan container images for known vulnerabilities using Trivy before deployment.
6) **Build repeatable scenario injection libraries and a quantitative evaluation harness.** Author versioned isolation and fairness violation scenarios with ground-truth logging, and a harness that computes precision, recall, false-positive rate, and detection latency per detector configuration.

V. Methodology and Results

A. System Architecture

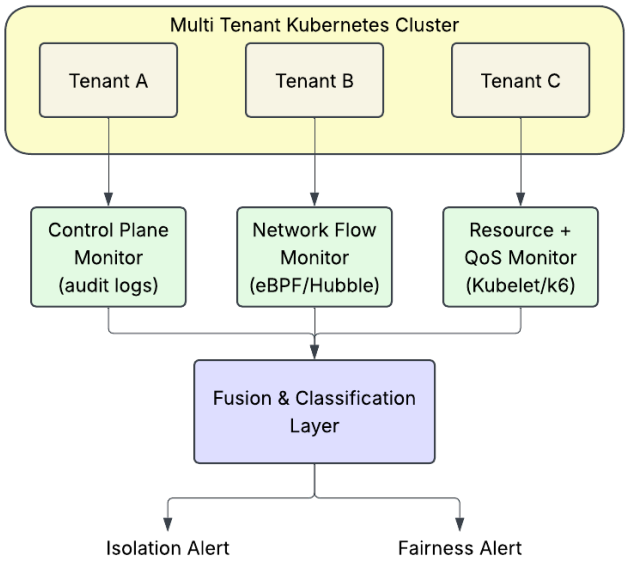


Fig. 1. System Architecture

B. Objective 1: Testbed Construction and Static Baseline

1) Testbed Construction: A local `kind` cluster was brought up with the default CNI disabled and Cilium installed in its place, with Hubble relay enabled for live eBPF-based network observability. Kubernetes API-server audit logging was wired to a host-readable `audit.log` via `kind extraMounts`. A demo FastAPI application was deployed across three tenant namespaces (`tenant-a`, `tenant-b`, `tenant-c`), each provisioned with:

- A **ServiceAccount** with a **Role/RoleBinding** scoped to read-only access within its own namespace only.
- A **default-deny NetworkPolicy** (Ingress and Egress) with explicit allows for same-namespace traffic and DNS only.
- A **ResourceQuota** (`limits.cpu: 2`, `limits.memory: 1Gi`, `pods: 10`) and a **LimitRange** with per-container defaults.

Boundaries were verified as effective: cross-namespace API calls are denied, cross-namespace traffic is dropped, and over-quota pod creation is rejected by the API server.

2) Static Baseline: kube-bench: `kube-bench` was run against the `kind` control-plane node using the CIS profile `cis-1.9` for Kubernetes 1.31:

Category	PASS	FAIL	WARN	INFO
Master	40	7	12	0
etcd	7	0	0	0
Control Plane	1	0	4	0
Node	16	2	6	0
Policies	2	4	29	0
Total	**66**	**13**	**51**	**0**

TABLE I
KUBE-BENCH CIS KUBERNETES 1.9 RESULTS

The Policies category (29 WARN) highlights kube-bench's key limitation. NetworkPolicy coverage, RBAC scoping, and namespace separation are all marked `Manual`/`WARN`. kube-bench can flag that a control point should be checked, but cannot confirm the specific policies are correct and in force at runtime.

3) Supplementary Check: Resource Quota Coverage: A supplementary script was written to fill a gap in kube-bench, which does not check for `ResourceQuota`/`LimitRange` objects at all.

Namespace	ResourceQuota	LimitRange	Result
tenant-a	✓	✓	PASS
tenant-b	✓	✓	PASS
tenant-c	✓	✓	PASS

TABLE II
SUPPLEMENTARY RESOURCE QUOTA CHECK RESULTS

C. Objectives 2–6: Implementation Status

Objectives 2 through 6 have all been fully implemented and smoke-tested against live cluster events, including the control-plane monitor, network-flow monitor, resource and QoS monitor, fusion and classification layer, CI/CD Trivy gate, 8 injection scenarios, and the full evaluation harness. A hand-run end-to-end test confirmed correct classification for one isolation and one fairness scenario. The full evaluation sweep, covering 8 scenarios $\times$ 5 repetitions plus 5 negative-control windows, is implemented and ready to execute. Quantitative results will be reported in the final submission.

VI. CONCLUSION

This report presents the mid-point implementation of a unified runtime detection system for tenant-isolation and QoS fairness violations in multi-tenant Kubernetes clusters. All six objectives have been implemented and smoke-tested. Objective 1's results, a kube-bench static baseline (66 PASS / 13 FAIL / 51 WARN) and a supplementary quota check (3/3 PASS), establish that the tenant isolation boundaries are real and enforced, and that static analysis alone is insufficient to continuously verify them at runtime. The remaining work is executing the full evaluation sweep to produce final precision, recall, false-positive rate, and detection latency results, which will be reported in the final submission.

REFERENCES

1) M. S. Shamim, F. Hu, and M. M. Rahman, "Dynamic Application Security Testing for Kubernetes Deployment: An Experience Report from Industry," in *Proc. ACM International Conference on the Foundations of Software Engineering (FSE)*, 2025.
2) Cilium Authors, *Cilium: eBPF-based Networking, Observability, and Security*, 2024. [Online]. Available: https://cilium.io
3) Cilium Authors, *Hubble: Network, Service & Security Observability for Kubernetes*, 2024. [Online]. Available: https://github.com/cilium/hubble
4) Aqua Security, *kube-bench*, 2024. [Online]. Available: https://github.com/aquasecurity/kube-bench
5) Falco Authors, *Falco: Cloud Native Runtime Security*, 2024. [Online]. Available: https://falco.org
6) Aqua Security, *Trivy: Vulnerability Scanner for Containers*, 2024. [Online]. Available: https://github.com/aquasecurity/trivy
7) The Kubernetes Authors, *Kubernetes Documentation*, 2024. [Online]. Available: https://kubernetes.io/docs